

第 1 页

Q1. Describe the echo algorithm using pseudo code.

题意（中文简述）：让你用伪代码描述分布式系统中 Echo 算法（又称“回声算法”）的逻辑流程。

A1. 答案与思路

❓ 算法核心

Echo

算法常用于在分布式系统中收集或传播某条信息，或用于树状结构的汇总问题。它通常由一个发起者（initiator）发送消息给所有邻居，然后通过“发起者收集响应”的方式完成。

❓ 基本思路

1. 发起者（initiator）向所有邻居发送一条消息，表示“我想要收集信息”或“请给我回声”。
2. 每个收到消息的节点，如果是第一次收到该消息，则会继续将该消息转发给它的所有邻居（除了发送者），并等待所有邻居的“回声”。
3. 当一个节点从它所有已转发消息的邻居处都收到了回声之后，才向它的父（即消息来源）发送回声。
4. 最终发起者收到所有邻居的回声后，整个算法结束。

❓ 伪代码示例

pseudo

复制编辑

```
procedure ECHO_Algorithm(Initiator):
```

```
    Initiator.parent := None
```

```
    send "ECHO_REQUEST" to all neighbors of Initiator
```

```
    Initiator.waitingFor := set of all neighbors
```

```
on receive "ECHO_REQUEST" from neighbor p at node v:
```

```
    if v.parent is undefined: // first time receiving
```

```
        v.parent := p
```

```
        v.waitingFor := all neighbors of v except p
```

```
        if v.waitingFor is empty:
```

```
            send "ECHO_REPLY" to p
```

```
        else:
```

```
            send "ECHO_REQUEST" to all nodes in v.waitingFor
```

```
    else:
```

```
        // If v.parent is already defined, do nothing (ignore duplicates)
```

```
on receive "ECHO_REPLY" from neighbor w at node v:
```

```
    remove w from v.waitingFor
```

```
    if v.waitingFor is empty:
```

```
if v.parent is not None:
    send "ECHO_REPLY" to v.parent
else:
    // v is initiator, so it knows the echo is complete
    algorithm_terminate()
```

Q2. Understand the deadlock detection schemes for the AND and OR model.

题意（中文简述）：理解并区分 AND 模型与 OR 模型下的分布式死锁检测策略。

A2. 答案与思路

② AND 模型

- 等待条件：进程需要同时等待所有资源（或所有依赖）才能继续执行。
- 死锁检测：通常使用“等待图(Wait- for Graph)” + 分布式探测算法（例如 Chandy 等）在整个系统内判断是否存在环。如果在 AND 模型下存在“闭环”，表示死锁。

② OR 模型

- 等待条件：进程只需等待其所依赖的一组资源中的任意一个即可继续。
- 死锁检测
：需要检测的是“不可满足的等待集合”，通常可以将其转换为一个“超边图”或使用合适的分布式探测算法来判断是否存在无法被满足的等待条件。例如在 OR 模型下，如果一组进程相互等待，但由于只需获得“其一”就能继续，可能并非死锁；只有当这一整组都无法获得任何所需资源时，才是真正死锁。

② 区别

- 在 AND 模型中，循环表示死锁；
- 在 OR 模型中，必须判断整个循环中是否没有可用的出路（某个资源能被满足），才是死锁。
- 实际实现中，OR 模型更复杂，需要多方面判断可用资源或可解除等待的可能性。

Q3. Outline an algorithm that finds the CPU load information on each of the machines hosting the tasks.

题意（中文简述）：在分布式系统中，每个任务只知道与它通信的任务，如何收集所有机器的CPU负载信息？

A3. 答案与思路

② 思路

1. 选出发起者
：任选一个任务作为汇总者（coordinator）或让它主动请求其他任务的信息。
2. 信息传递：
② 该发起者向自己直接知道的所有任务发送“请求CPU负载信息”的消息。

②

每个收到请求的任务，如果尚未上报过负载信息，就将自己的负载信息和“已知的其他任务地址”一并返回给发起者，并同时向那些尚未接触过的任务继续扩散请求。

3. **避免重复/循环**：在每个节点上维护一个“已报告”标记或一个全局唯一标识的查询 ID，用来避免重复处理。

4. 汇总

：所有节点的负载信息最终会沿着消息链路汇总回发起者。发起者收集到全部信息后，完成查询。

② 伪代码要点

- 使用类似 Echo 或 Flooding + 收敛的机制，每个节点只处理一次请求并把信息向上传递。
- 具体实现可以参考 Echo 算法的结构，只是传递负载信息而已。

Q4. Modify Chandy's deadlock detection for the AND model so that, when a deadlock is detected, the process with the smallest ID can be aborted.

题意（中文简述）：在 AND 模型下使用 Chandy 算法检测死锁后，如何在发现死锁时中止 ID 最小的进程来解除死锁？

A4. 答案与思路

② 原有 Chandy 算法（AND 模型死锁检测）

- 通过分布式标记、Probe 消息等判断是否存在无法推进的循环等待。

② 改动

0. **唯一 ID**：假设每个进程都有唯一 ID。

1. **检测出死锁后**

：在传统算法中，探测方（发起死锁检测的进程）可能会收集到一个循环中的所有进程信息。

2. **选择最小 ID**：维护一个全局最小 ID（在检测的过程中即可比较）。

3. **通知中止**：当检测端确认死锁成立后，通过消息通知环中所有节点：“中止 ID 最小的那个进程”，或直接发送“Abort 进程X”的命令给该进程。中止操作会释放资源，从而打破死锁。

② 要点

- 在死锁探测消息（Probe）中携带一个字段，用来记录到目前为止发现的最小进程 ID。
- 当发现循环或判定死锁时，该最小 ID 即为要中止的进程。
- 确保在中止后，其他进程也能得知这一情况，从而从等待状态中恢复。

第 2 页

Q1. What problem does the reference counting garbage collection algorithm might have in a distributed system?

题意（中文简述）：引用计数法在分布式系统中可能遇到的问题是什么？

A1. 答案与思路

② 主要问题：

1. **循环引用**：如果对象 A 和 B 互相引用，对象就不会被回收。
2. **网络延迟/消息丢失**
：在分布式环境中，引用计数的增减需要跨节点更新，一旦丢包或延迟，计数就会不准确。
3. **同步代价**：频繁更新引用计数需要大量同步操作，代价高。

② 在分布式系统中特别突出的就是**丢失引用更新或者循环依赖**，导致垃圾对象无法被回收。

Q2. Describe how the weighted reference counting garbage collection algorithm works.

题意（中文简述）：描述加权引用计数算法如何运作。

A2. 答案与思路

② 加权引用计数算法（Weighted Reference Counting）

- 将引用计数拆分为若干权重（weight），当对象的引用被复制或转移时，相应地分摊或转移权重。
- 当权重为 0 或对象不再被任何有效引用持有时，对象可被安全回收。
- **关键点**
：每个引用并不直接增加“1”到计数，而是按照一定的规则分配权重，使得丢失一次更新不会导致整个计数混乱，并且可以正确回收循环。

② 简要流程：

0. 初始对象有一个引用时，会分配一个初始权重（如 1）。
1.
如果要复制这个引用，那么会将现有权重的一部分“分给”新引用，并减少原引用相应的权重。
2. 当一个引用失效时，就释放它的权重。若对象所有引用总权重变为 0，回收该对象。
3.
这样可以更好地应对分布式环境下的并发与复制问题，不会出现传统引用计数的单纯“+1”或“-1”不平衡导致的混乱。

Q3. Why does the weighted reference counting garbage collection algorithm not have the problem encountered by the reference counting garbage collection algorithm?

题意（中文简述）：为什么加权引用计数算法不会遇到普通引用计数的那些问题？

A3. 答案与思路

② 原因

1. 拆分权重

：它将“计数”拆分为多个部分，在不同副本或节点之间分配；即使丢失一部分更新，整体也能保持一致性。

2. 处理循环

：通过权重转移和检测，可以识别环中的对象引用是否真的活跃；若环中没有有效的外部权重输入，最终权重会归零，从而被回收。

3. 消息鲁棒性

：加权引用计数在分布式环境下，允许引用增减操作异步进行，不容易因为某一次消息丢失就彻底出问题。

Q4. Understand how the algorithm for detecting cyclic garbage works.

题意（中文简述）：理解如何检测循环垃圾的算法。

A4. 答案与思路

② 循环垃圾检测：

- 当对象之间相互引用，没有外部可达性时，就形成“垃圾环”，普通引用计数算法无法回收。
- 一种典型做法：利用分布式标记-清除思想或合并了加权引用计数的循环探测机制，对可达性做一次专门扫描；若对象只能在环中相互引用，而没有其他活引用，判定为循环垃圾。

② 简要步骤：

0. 选取触发节点，发送探测到环中对象。

1. 对象收到探测后

，继续向它的引用发送探测，若最终探测没有从外界获得活引用，则判定是循环垃圾。

2. 回收 环中对象均可回收。

Q5. In the algorithm for collecting cyclic garbage, how do we know whether the probes have reached an object through all the

predecessors of the object?

(Hint: the algorithm works as a complement of other garbage collection algorithms, e.g., weighted reference counting.)

题意（中文简述）：在循环垃圾收集算法中，如何确认探针已经通过对象的所有前驱节点到达该对象？

A5. 答案与思路

② 要点

1. 在探测消息中携带一个计数或集合，用来记录已经访问过的路径或前驱节点。
2. 当对象收到探测时，会等到从所有前驱都收到探测或确认后，才认定“来自所有前驱的探测都已到达”。
3. 也可结合加权引用计数：当对象的剩余权重都来自同一轮探测，就说明没有外部未探测到的引用。

② 核心思路：

- 通过在探针中累积信息，或使用“等待前驱数”的技术。
- 当对象确认所有前驱节点都传来了探测，则证明已覆盖全部可能来路。

Q6. Use the principles of the weighted reference counting to develop an algorithm that finds a set of nodes in a wait-for graph such that the outgoing edges from the nodes in this set always end on nodes in this set. (It would be possible to detect deadlock in the OR deadlock model using this algorithm.)

题意（中文简述）：用加权引用计数的原理，找出等待图（wait-for graph）中某节点集合，使得从集合内节点发出的所有边仍然指向集合内的节点（用于 OR 模型死锁检测）。

A6. 答案与思路

② 思路

- 可以把“加权引用计数”类比为“等待边的权重”。若一组节点之间的等待依赖都在这组节点内部，而且没有外部的“活”引用/等待边进入或离开这组，则这组节点形成一个“闭合等待子图”，意味着潜在的死锁（尤其适用于 OR 模型下，没有可用的外部资源可以解除等待）。
- 步骤
 1. 初始时为每条等待边分配一个权重。
 2. 当某个等待可以被满足（相当于引用失效或释放），则把相应权重减少。
 3. 如果最终发现某个子集的权重总是自给自足（既无外边流出，也无外边流入可被

满足)，说明这个子集是封闭的，可能就是死锁所在。

❓ 要点

0. 在 OR

模型中，只有当所有节点都无法等待到外部的可用资源时，才是真正的死锁；这可以通过检查子图内是否有“出口边”指向能被满足的等待。

1. 若这些出口边全部指向集合内节点，且集合内节点无法得到外部资源，则判定死锁。

第 3 页

Q1. Understand the meaning of FIFO, total and causal order multicast and use an example to show why they are important.

题意（中文简述）：理解 FIFO、多播全序（total order）、以及因果序（causal order）多播的意义，并举例说明它们的重要性。

A1. 答案与思路

1. FIFO（先进先出）多播

- 同一发送者发送的消息，接收者必须按发送顺序接收。
- **重要性** 保证某个进程自己发出的消息顺序不乱。

2. 全序（total order）多播

- 系统中所有进程看到的消息顺序都一致。
- **重要性** 例如分布式日志、分布式数据库更新时，必须保证所有节点的事件顺序一致。

3. 因果序（causal order）多播

- 如果消息 m1 的发送发生在消息 m2 的发送之前（即 $m1 \rightarrow m2$ ），则系统中任何进程都不能先收到 m2 再收到 m1，须保持因果顺序。
- **重要性** 保证消息之间的因果关系不被颠倒。

❓ **示例** 在一个分布式聊天系统中，若没有

FIFO，多播可能导致一个用户先发的消息在其他人看来反而是后到；若没有因果序，回复可能在消息之前显示；若没有全序，则多个用户的并发操作顺序可能不一致，从而引起数据冲突。

Q2. Understand the three algorithms for implementing the three ordered multicast protocols.

题意（中文简述）：理解实现上述三种多播协议（FIFO、因果、全序）的算法。

A2. 答案与思路

1. FIFO 多播

- 每个发送者维护一个本地发送序号 (sequence number), 接收方按 <发送者ID, 序号> 排序来重排消息, 保证同一发送者的顺序。

2. 因果多播

- 使用向量时钟 (Vector Clock) 或部分排序逻辑时钟;
- 只有当所有因果先行消息都被接收后, 才交付当前消息。

3. 全序多播

- 有多种实现方式, 例如:
 - ② **集中式序列器**: 由一个进程给所有消息分配全局序号;
 - ② **Ring 协议**: 进程按照环形结构为消息排序;
 - ② **协议如 ISIS 中的 ABCAST**
: 基于组内确定性算法为消息达成共识并分配全局顺序。

Q3. Can the causal order protocol be used to ensure FIFO order multicast? Explain why.

题意 (中文简述): 因果序协议能否保证 FIFO 多播?

A3. 答案与思路

- ② **答案** 是的, 因果序可以同时保证 FIFO 顺序。
- ② **原因** 因果序中, 如果一个进程发送了两条消息 m1 与 m2, 其中 m1 的发送因果在先, 那么接收者一定会先收到/ 交付 m1, 再收到/ 交付 m2。这就包含了 FIFO 对同一发送者的顺序要求。

Q4. Can the FIFO order multicast protocol be used to ensure total order multicast? Explain why.

题意 (中文简述): FIFO 协议能否保证全序多播?

A4. 答案与思路

- ② **答案** 不能, 仅有 FIFO 不足以保证全局一致顺序。
- ② **原因** FIFO 只保证同一发送者的发送顺序, 而不同发送者之间的并发消息顺序并没有强制要求; 因此无法保证系统所有节点上对并发消息的全局一致排序。

Q5. Design an algorithm that uses the partially ordered logical timestamp to ensure the causal-ordering of multicast messages.

题意（中文简述）：设计一个使用部分排序逻辑时间戳来保证因果多播顺序的算法。

A5. 答案与思路

② 常见做法：向量时钟 (Vector Clock)。

1. 系统中每个进程维护一个向量时钟 $VC[i]$ 。
2. 每次发送消息前，先执行 $VC[i] := VC[i] + 1$ ，并把 VC 附在消息中。
- 3.

接收者收到消息后，先将本地向量时钟与消息的向量时钟逐项取最大值，然后再将消息放入缓冲区；只有当该消息的所有因果前驱都被交付（即本地时钟中，对所有 j ， $VC_local[j] \geq VC_msg[j]$ ），才正式交付消息。

② **关键** 通过向量时钟的比较条件 ($\forall j: VC_local[j] \geq VC_msg[j]$)

来判断因果依赖是否已经满足，从而保证因果顺序。

Q6. What properties does virtual synchrony have?

题意（中文简述）：虚拟同步 (Virtual Synchrony) 具有什么性质？

A6. 答案与思路

② **虚拟同步** 典型性质：

1. **View 一致性**：当发生组视图 (group view) 变化（如某进程加入、离开或失败），所有存活进程必须以一致顺序看到同样的视图变化。
2. **消息一致性**
：在一次视图改变前已发送但未被交付的消息，要么在新视图中被所有进程交付，要么被所有进程丢弃，保证一致性。
3. **有序性** 通常与全序 (total order) 或因果有序结合。

第 4 页

Q1. The implementation of the virtual synchrony discussed in our lecture assumes that no failure occurs during the view change. Augment the scheme so that it can cope with failures during the view change.

题意（中文简述）：在视图变更期间 (view

change) 也可能发生节点故障，如何扩展虚拟同步协议以应对这种情况？

A1. 答案与思路

② 基本做法：

1. 在进入新的视图之前，进程会进行一轮确认（e.g. Flush 阶段），保证上一视图的消息都达成一致（要么所有存活进程都交付，要么都丢弃）。
2. 若在这期间有新的故障发生，需要**重新启动**视图变化协议，更新视图成员列表；并对“未确认/未交付”消息做一致性处理。
3. 可能需要“多轮确认”或“冗余协调者”的方式来确保即便在视图改变过程中也能检测到故障并刷新视图。

② 要点

○

视图变化不再只是一次简单的阶段，而是若在该阶段又发现成员故障，则要立即启动下一次视图变化，直到所有存活节点都对某一最终视图达成一致。

Q2. Assume that multiple processes can contact different processes in the process group concurrently for joining the process group. Augment the scheme discussed in our lecture to handle the view change correctly.

题意（中文简述）：多个进程同时加入组，需要如何修改虚拟同步算法来正确更新视图？

A2. 答案与思路

② 关键

1. 当多个新进程同时请求加入，需要一个**“视图安装”协调者**或“分布式共识”来决定新视图。
2. 可以将所有新加入请求先放入一个队列，然后统一执行一次视图更新，生成新的视图号。
3. 若有并发请求，可对请求进行合并处理；或者采用类似分布式锁的机制，保证对“视图更新”操作的原子性，避免出现多个不同版本的视图冲突。

② 实现

- 类似于组通信系统中采用的 **membership service**，确保对组成员列表的更新序列化（total order），让所有存活进程看到相同的一致视图序列。

Q3. Extend the implementation of the virtual synchrony discussed in our lecture to provide reliable total-order multicast.

题意（中文简述）：在虚拟同步的基础上，如何进一步提供可靠的全序多播？

A3. 答案与思路

② 常见思路：

1. 在组内结合 Virtual Synchrony + 全序协议（如：使用组内序列号、组内一致性协议）。
2. 每个消息分配一个全局序号（可以由组内的“发号器”或使用一致性算法共识出下一个序号）。
3. 在视图不变的情况下，所有进程使用相同的序号来交付相同的消息。
4. 若视图发生变化，也通过虚拟同步保证要么所有已发送消息都被交付，要么被丢弃，然后切换到新视图继续分配序号。

② 要点

- 通过可靠组播（reliable broadcast）和全局序号分配，结合 VS 视图机制，达到可靠全序多播。

第 5 页

Q1. Use pseudo code to describe Ricart and Agrawala's algorithm.

题意（中文简述）：使用伪代码描述 Ricart 和 Agrawala 算法。

A1. 答案与思路

② Ricart & Agrawala

：分布式互斥算法，只有当一个进程得到所有其他进程的许可（Reply）后才能进入临界区。

② 伪代码要点（简化示例）：

```
pseudo
复制编辑
// Each process i has:
timestamp := 0
state := RELEASED // can be RELEASED, WANTED, or HELD
deferred_replies := empty set
// On event request to enter critical section:
state := WANTED
timestamp := timestamp + 1
request_ts := timestamp
```

```

broadcast(Request(i, request_ts)) to all other processes
wait until (received Reply from all other processes)

// Enter critical section
state := HELD
// ... critical section ...
state := RELEASED

on receive Request(j, ts_j):
    timestamp := max(timestamp, ts_j) + 1
    if (state == HELD) or
       (state == WANTED and (ts_j, j) > (request_ts, i)) then
        // Defer reply
        add j to deferred_replies
    else
        send Reply to j

on receive Reply(j):
    // Mark that j has replied

// After exiting critical section:
for each p in deferred_replies:
    send Reply to p
clear deferred_replies

```

② 比较关键

：若自己想进临界区且自己的请求“优先级”（时间戳、小ID）更高，则暂缓回复对方，否则立即回复。

Q2. Explain why Ricart and Agrawala's algorithm satisfies the safety, liveness and fairness properties.

题意（中文简述）：为什么该算法满足安全性（互斥）、活性（不会无限等待）以及公平性（先请求先服务）？

A2. 答案与思路

② 安全性 (Mutual Exclusion):

所有进程必须得到所有其他进程的同意才能进入临界区；同一时刻不可能有两个进程都拿到所有其他进程的许可，从而保证互斥。

② 活性 (Liveness):

请求最终会得到回复，且没有进程会永久拒绝他人的许可。一旦持有临界区的进程退出后，会发送回复给所有延迟请求进程，故不会出现无限等待。

② 公平性 (Fairness):

使用 (时间戳, 进程

ID) 进行排序, 请求时间早的优先进入临界区, 较晚的要等先请求者完成, 避免饥饿。

Q3. Explain why Maekawa's algorithm satisfies the safety, liveness and fairness properties.

题意 (中文简述): Maekawa 算法如何保证安全、活性与公平?

A3. 答案与思路

② Maekawa 算法

: 将系统划分为若干个“投票组(Quorum)”, 每个进程进入临界区需要获取其所属“投票组”中多数成员的许可。

② 安全性

○

不同进程的投票组存在交叉 (任意两组至少有一个公共成员), 因此两个进程不可能同时拿到各自组内大多数投票。

② 活性

○

投票算法中, 每个进程请求时, 组内成员只要没投票给其他进程或已收回投票, 就会投票给请求者; 在合适的调度和无故障下, 不会无限等待。

② 公平性

○

若采用合适的队列策略 (如 FIFO) 来分配投票, 则先请求的进程先得到许可, 并在使用结束后释放投票, 从而让后续请求者获得机会。

Q4. Assume that a task might crash in Raymond's algorithm. Augment Raymond's algorithm as described in the lecture to make it tolerate task crash.

(Hint: after a task recovers from a failure, it can reconstruct its request_q and holder by asking for the states of its neighbours)

题意 (中文简述): 在 Raymond 算法中, 如果节点崩溃并恢复后, 如何让它重建 token_request 队列和 holder 信息?

A4. 答案与思路

② Raymond's 算法核心: 维护一个逻辑上的“树”或环结构, token

在拥有者和请求者之间沿着树边移动; 每个节点有一个 holder 指针指向 token 的位置, 还有一个请求队列 request_q。

② 增强容错：

1. 故障检测

：如果一个节点发现自己邻居故障，则可能需要更新树结构或将请求转发给其他节点。

2. 恢复时 崩溃节点向邻居请求它们对自己的 request_q 和 holder

状态副本，从而重建本地信息；

3. 若该节点原本持有 token，则需要在系统中找回 token

的真实拥有者；可能由邻居或全局机制来判定当前 token 所在。

4.

要保证在恢复期间不破坏已有的互斥控制，可暂时阻塞该节点对新的请求操作，直到它的状态恢复完毕。

Q5. In Raymond's algorithm, why does the Make_request routine only send the token_request message to a task's neighbour if the task has not sent a token_request message to the neighbour?

题意（中文简述）：为什么在 Raymond 算法中，每当请求 token 时，如果已经向某邻居发送了 token_request，就不再重复发送？

A5. 答案与思路

② **原因** 为了避免重复请求引发消息风暴，也避免队列重复。

② **Raymond 逻辑**：一旦你向某个邻居发出了请求，该邻居就会在它的 request_q 中记录你的请求；如果你再次发送相同请求，会导致重复排队或无意义的重复处理。

② **效率** 只需等待邻居将请求逐级向 holder 方向转发；重复发送只增加网络负担。

Q6. Assume that you are developing a distributed editing system... Design a scheme that ensures a file can only be modified by one user at a time.

题意（中文简述）：无中心服务器，文件分布在各用户本地，如何确保同一时刻只能一个用户修改？

A6. 答案与思路

② **可行方案**：使用分布式互斥（类似 Ricart & Agrawala 或 Maekawa 或 Raymond 算法）来对每个文件的访问进行控制。

1. 当用户想编辑文件时，需要先在分布式系统中发出“想要编辑文件X”的请求。

2. 其他用户进程若没在编辑该文件，就同意请求。若有人在编辑，则等待对方释放。

3. 获得所有必要的许可后，该用户才能进入编辑状态；编辑完毕后广播“释放”。

② **要点**

- 每个文件可以维护一个独立的分布式互斥流程，也可以为每个文件维护一个逻辑“token”。持有 token 的用户才可以编辑。
- 没有中心服务器时，可以用环形令牌(Ring Token)或其他去中心化算法来进行调度。

第 6 页

Q1. Why is obtaining a consistent global snapshot difficult in a distributed system? Use an example.

题意（中文简述）：为什么分布式系统中获取一致全局快照不易？请举例说明。

A1. 答案与思路

② 原因

1. **并发** 不同进程同时进行本地操作，状态随时变化；
2. **无全局时钟**：无法一瞬间冻结所有进程去拍照；
3. **消息在通道中**：进程快照可能记录了消息的发送却没记录接收，或者反之，导致不一致。

② 示例

- A 向 B 发送转账 10 元，若全局快照在 A 减少 10 元后拍到，但在 B 增加 10 元前拍到，就出现“丢失”或“不一致”的问题。

Q2. What are the conditions that need to be satisfied to make a global state consistent?

题意（中文简述）：一致全局状态需满足哪些条件？

A2. 答案与思路

② Chandy-Lamport 定义：

- **一致性条件**
：若在快照中看见一个消息在通道中，则必须在发送方的快照中记录该消息已经被发送；若在接收方的快照中记录了消息已经接收，则在快照中也要记录该消息已被发送。
- 换言之，不能出现“接收但未发送”或“发送但通道/接收都未反映”的矛盾状态。

Q3. For some systems, the Chandy-Lamport algorithm can be modified so that all the channel states are recorded as empty... Which property do these systems need and how to modify the

algorithm?

题意（中文简述）：在某些系统中，可以把所有通道状态当作空来简化，系统需要具备什么特征，以及如何修改？

A3. 答案与思路

② 需要的特性：消息传输FIFO

且发送端在发送后不会重发，或者系统保证消息在一个已知时间界内必达，从而在标记开始到结束之间的任何消息都能被确定归属到某个快照中。

② 如何修改：

1.

当一个进程收到“快照标记”后，清空其输入通道中的所有未处理消息（或将其归属于本次快照之前），并且不再记录后续到达的消息到该通道状态里。

2. 这样可将通道状态直接视为“空”。

- 但前提是不会丢失消息，且消息顺序可保证不会与快照界限交错。

Q4. Modify the Chandy–Lamport algorithm so that it handles the case in which several sites initiate the snapshot algorithm simultaneously, minimizing the exchanged messages.

题意（中文简述）：多个站点同时发起快照，如何最小化消息？

A4. 答案与思路

② 思路

1. 当一个节点收到多个快照标记时，如果它已经在某个标记 ID 下进行了快照，则无需重复做快照，可以使用同一个快照结果或将其合并。
2. 只在第一次收到某个标记 ID 时才向通道发送标记，后续相同 ID 的标记可忽略；
3. 若不同 ID 的标记同时出现，可以为每个快照 ID 分别记录本地状态，但仍可共享某些通道的状态或对其进行合并，以减少重复发送标记。

② 要点

- 在标记消息里携带一个“快照 ID”；
- 每个节点记录已处理过的“快照 ID”，避免重复操作和消息。

第 7 页

Q1. Describe the two-phase commit (2PC) protocol using an example.

题意（中文简述）：举例描述两阶段提交 (2PC) 协议。

A1. 答案与思路

1. 准备阶段 (Phase 1: Prepare)

- 协调者 (coordinator) 向所有参与者发送“准备提交 (prepare)”请求，等待其回应。
- 参与者检查是否能提交（如本地事务是否成功、资源是否就绪），若能则答复“可以提交 (vote commit)”，否则“不能提交 (vote abort)”。

2. 提交阶段 (Phase 2: Commit/Abort)

- 若协调者收到所有参与者的“可以提交”，则向所有参与者发送“commit”命令；
- 如果有任一参与者答复“不能提交”，或出现协调者超时，则协调者发送“abort”命令。
- 参与者接到最终命令后，执行提交或回滚，并返回完成确认。

② 示例

：银行转账跨多个分支行，协调者检查每个分支行是否能完成扣款/存款操作，均OK后再发出“提交”命令，否则取消操作。

Q2. Explain why the 2PC protocol might block under certain failure conditions and discuss the possible solutions.

题意（中文简述）：为什么 2PC 在某些故障情况下会阻塞？有哪些可能的解决方案？

A2. 答案与思路

② 阻塞原因：

- 在“准备阶段”结束后，如果协调者崩溃且未告知最终结果，而参与者已经表态“准备提交”，此时就不知道是应该提交还是回滚，陷入阻塞状态。
- 参与者会等待协调者的最终决策，若协调者一直不恢复或消息丢失，就无法继续。

② 解决方案：

0. 超时 + 协议升级：如采用 **三阶段提交(3PC)** 或使用某种故障检测 + 选举新的协调者；
1. 仲裁或合并：采用一致性协议（如 Paxos/ Raft），由多个协调者副本来决定结果，避免单点故障。

Q3. What is the three-phase commit (3PC) protocol? Discuss how it addresses the blocking problem in 2PC.

题意（中文简述）：什么是三阶段提交 (3PC)，它如何解决 2PC 中的阻塞问题？

A3. 答案与思路

② **3PC 概念**：在 2PC 基础上插入一个额外阶段（通常称为“prepare to commit”和“pre-commit”分离为两个阶段），使得参与者有机会在协调者故障时也能达成一致决定，减少阻塞。

② **阶段**

1. **CanCommit**（询问是否可提交，相当于 2PC 的第一阶段）。

2. **PreCommit**（在真正的 Commit

之前，让参与者进入“准备提交但未正式提交”状态，如果此时协调者故障，参与者可根据这一步的状态来决定最终提交还是回滚）。

3. **DoCommit**（最终提交）。

② **解决阻塞的原理**：

○

增加的中间阶段提供了超时检测和一致性判断机制；若协调者故障，参与者可以根据自己所处的阶段，协商一个统一的结果（如果还在 PreCommit 状态且确认大多数也在此状态，可以继续提交，否则回滚），从而不会无限阻塞。

Q4. What is the difference between the termination protocol in 2PC and 3PC?

题意（中文简述）：2PC 与 3PC 的终止协议（termination protocol）有何不同？

A4. 答案与思路

② **2PC**

：当协调者失败后，参与者只能等待协调者重启或超时后自行判断，但由于只有两阶段，若不知道其他参与者的投票，容易出现阻塞。

② **3PC**：有明确的 **PreCommit** 状态；在协调者失败后，若大多数参与者都已经进入

PreCommit，则可以决定提交；若都还在等待阶段则回滚。这样通过多一步状态，减少对单个协调者的依赖，使得**终止协议**可以在参与者间也达成结果，避免长时间阻塞。

Q5. How can we handle network partitions in the two-phase commit protocol?

(Hint: you may need to augment the protocol or add heuristics.)

题意（中文简述）：在 2PC 协议中如何处理网络分区？

A5. 答案与思路

② **网络分区**：系统被分隔成互不连通的区域；协调者与部分参与者无法通信。

② **处理方法**：

1. **超时 / 尝试重新连接**

：若参与者没收到协调者的指令，可以在超时后放弃或回滚；但这可能导致一致性风险，需

要谨慎。

2. 仲裁点

：在分区情况下，让包含仲裁或大多数节点的一方继续处理事务，另一方中止或阻塞。

3. 复制协调者：采用多副本协调者，若一个副本所在网络分区，则由其他副本继续协议流程。

- ② **启发式** 有的实现中选择“保守”策略——如果网络分区，就尽量先回滚来保证一致性；或采用 Paxos 等共识协议来决定提交还是回滚。

Q6. Explain how to incorporate replicated coordinators into the two-phase commit protocol to improve fault tolerance.

题意（中文简述）：如何在 2PC 中加入多个协调者副本以提高容错？

A6. 答案与思路

② 思路

1. 将协调者角色在多个节点上复制（类似主备 or 多主）。
2. 通过一致性协议（Paxos/ Raft）来确定“谁是当前有效协调者”，并且在关键的写操作（如事务决定）时达成副本一致。
3. 如果当前协调者故障，其余副本可快速接管并继续完成协议，而不会造成长时间阻塞。

Q7. Assume that you are implementing a distributed database system. Propose an algorithm that ensures a transaction either commits in all sites or aborts in all sites, and argue why your solution guarantees consistency.

题意（中文简述）

：实现分布式数据库时，如何保证事务要么在所有节点都提交，要么都回滚，并说明为什么能保证一致性？

A7. 答案与思路

② 可行方案：

1. **2PC/3PC**：典型的分布式原子提交协议。
2. **使用共识协议**：如 Paxos/ Raft 来达成“事务最终状态（commit or abort）”的全局一致决定。

② 一致性保证：

- 在 2PC 中，只有在所有参与者都同意的情况下才会进入提交阶段，否则任何一个投否决票或出现故障超时，都会导致整体回滚，从而实现原子性。
- 如采用 Paxos/ Raft，则保证了只有达成大多数共识后才会广播“commit”；任何子集都不能

单方面决定提交或回滚。